Assignment 1. Product Category Classification 1

DATA304: Big Data Analysis

Due Date: September 30, 2025

General Instructions

The programming assignments in this course are designed to foster your ability to implement and apply the concepts introduced during class. All datasets and resources for the assignments are provided at the following link: **Link**. You do not need to download new data for each assignment. Instead, simply place the newly provided Jupyter notebook files for each assignment into your existing `assignment_release` directory (or into the renamed directory, if you have changed it).

- **How to submit:**
  - You need to submit your code via LMS for all assignments. For each assignment, submit **All** the released `.ipynb` files, including all modifications you have made.
  - Assignments #1, #2, and #3 also require Kaggle submissions.
  - Assignments #4 and #5 do not require Kaggle submissions.

  Make sure you submit both LMS and Kaggle on time to receive the credit. All submissions are due by midnight (23:59 KST) on the deadline day.

- **How the assignments are graded:**
  - For Assignments #1, #2, and #3, you will receive full credit as long as you outperform our baseline code.
  - For Assignments #4 and #5, you will receive full credit as long as you submit your code on time.

  This policy is intended to encourage you to solve the problem by yourself and to compare your results with others. Use the Q&A board on LMS for all assignment-related questions.

Assignment Instruction

This assignment consists of two Jupyter notebooks: `P1a.data-exploration.ipynb` and `P1d.introduction-TF-IDF.ipynb`.

P1a: Data Exploration

In this part, we will explore the (simplified) E-commerce dataset to understand the information provided in each file. Data exploration is more than just loading the files — it allows us to examine the structure of the inputs and outputs, and prepares us for building retrieval and classification models later. Think of this step as the process of getting familiar with the dataset. By thoroughly understanding it in advance, we can make more informed decisions in the following stages, such as preprocessing, model design, and evaluation.

⌨ **Your Task:** Understand the given dataset and the associated tasks. We also recommend that you go beyond the provided examples and try various approaches to become more familiar with the data. No submission required.

P1d: Product Category Classification with TF-IDF Vectors

In this part, you will experiment with two approaches:

1. [**Part A**]: TF-IDF similarity between product text and category labels
2. [**Part B**]: TF-IDF vectors combined with a linear classifier

These two methods illustrate both a simple similarity-based classification approach and a training-based model. It is strongly recommended that you carefully follow and fully understand this guideline, as it will be essential for completing the subsequent tasks.

[Part A]: TF-IDF similarity between product text and category labels

In this first attempt, we will use a lexical similarity approach based on TF-IDF. The idea is straightforward: represent both product descriptions and category labels as TF-IDF vectors, and then compute their similarity (e.g., cosine similarity). This method does not involve learning parameters — it simply measures surface-level word overlap. While limited, it provides a simple baseline to check whether lexical features alone are sufficient for product classification.

⌨ **Your Task:** Carefully read and understand the provided code. Examine intermediate values by printing them out, and make sure you clearly understand what operations are being performed at each step and how they fit together in the overall process. No submission required.

[Part B]: TF-IDF vectors combined with a linear classifier

In this second approach, we go beyond simple lexical similarity and use a supervised model. Here, each product text is represented as a TF-IDF vector, which is then fed into a linear classifier.

Formally, given an input feature vector $\mathbf{x} \in \mathbb{R}^d$, the model computes:

$$\mathbf{z} = W\mathbf{x} + \mathbf{b}, \quad \hat{\mathbf{y}} = \text{softmax}(\mathbf{z}),$$

where $W \in \mathbb{R}^{C \times d}$ and $\mathbf{b} \in \mathbb{R}^C$ are the parameters of the linear layer, and C is the number of classes. The output $\hat{\mathbf{y}}$ is a probability distribution over the C possible classes.

The loss is then computed using the predicted distribution $\hat{\mathbf{y}}$ and the true label y[1]:

$$\mathcal{L} = -\sum_{c=1}^{C} \mathbf{1}[y = c] \log \hat{y}_c.$$

Here, $\mathbf{1}[y = c]$ is an indicator function that equals 1 only for the ground-truth class y, otherwise 0. This loss function is designed so that the model increases the predicted probability for the ground-truth class. If the model assigns a low probability to the correct class, the loss becomes large; if it assigns a high probability, the loss becomes small. This is what we call the cross-entropy loss.

The model learns to map TF-IDF features to the correct product category by training on labeled data. Unlike the similarity-based method, this approach can capture more complex decision boundaries, making it more flexible and potentially more accurate.

⌨ **Your Task:** Carefully read and understand the provided code. Examine intermediate values by printing them out, and make sure you clearly understand what operations are being performed at each step and how they fit together in the overall process. No submission required.

Improving Training with Validation Logic

To train a reliable model, it is not sufficient to only monitor performance on the training set. A separate *validation set* is needed to evaluate how well the model generalizes to unseen data. Without validation, the model may simply memorize the training data and appear to perform well, while actually failing on new examples (i.e., overfitting).

In this assignment, we provide a validation set that corresponds to 20% of the training data. You will use this set to monitor accuracy at the end of each epoch. By comparing validation performance across epochs, you can determine whether the model is improving in a meaningful way.

With the validation set in place, you can implement two key techniques:

- **Best model tracking:** Save the model state whenever the validation accuracy improves, so that you can later restore the most effective version of the model.
- **Early stopping:** Stop training when the validation accuracy does not improve for a specified number of epochs (*patience*), preventing wasted computation and reducing overfitting.

Through validation monitoring, best-state tracking, and early stopping, you will build a more efficient and generalizable training process.

⌨ **Your Task:** Now, your task is to improve the given training loop by implementing validation logic, and submit your best results to Kaggle.

[1] In PyTorch, the function `F.cross_entropy` already includes the softmax operation internally before computing the loss. Therefore, you should pass the raw logits $\mathbf{z} = W\mathbf{x} + \mathbf{b}$ directly, without applying softmax yourself.

Submission Guidelines

Kaggle Submission

- Submit your prediction file in `.csv` format to Kaggle: **Link**
- Ensure that the filename of your submission follows the format: `studentID_assignment1.csv`
- Set your **Kaggle team name** to your AWS account ID (e.g., `korea-dt-xx`). **This is mandatory in order to receive credit in our grading system. Please be careful!**
- There is **no limitation on the number of Kaggle submissions**.

LMS Submission

- Submit your source code (all `.ipynb` files) to the LMS.
- Make sure the submitted code can be executed without errors.